Serverless Data Cleaning Pipeline on AWS

Cloud Computing Methodologies

Name	Register No.
Sudharshini Manikandan	24MID0135
Sathya Sai S	24MID0112

Abstract

Organizations increasingly rely on data-driven decision-making, yet the raw data collected from multiple sources is often incomplete, inconsistent, or improperly formatted. Manually cleaning such data before analysis is time-consuming and does not scale as data volume grows. This project proposes a fully serverless, event-driven data cleaning pipeline built on Amazon Web Services (AWS), leveraging Amazon S3, AWS Lambda, and Amazon CloudWatch. Whenever a raw dataset is uploaded to a designated S3 bucket, an S3 event automatically triggers a Lambda function that performs data cleaning operations such as null-value removal, duplicate elimination, and data-type standardization using the pandas library. The cleaned dataset is then stored in a separate output bucket, and processing logs are captured in CloudWatch for monitoring and auditing. The project demonstrates key cloud computing principles including serverless computing, event-driven architecture, and least-privilege security using AWS Identity and Access Management (IAM), while remaining entirely within the AWS Free Tier.

Problem Statement

Raw datasets deposited into organizational data lakes are often structurally compromised. When data analysts attempt to query this unclean data, it results in skewed analytics, processing errors, and corrupted downstream machine learning models. The conventional approach to resolving this involves provisioning dedicated compute infrastructure (such as standard servers) to run scheduled batch-cleaning jobs. This creates two distinct problems: first, the business incurs unnecessary idle compute costs while waiting for data to arrive; second, there is a significant time delay between data ingestion and data availability. There is a critical enterprise need for a lightweight, zero-maintenance pipeline that can automatically and instantaneously sanitize data payloads the moment they breach the storage perimeter, ensuring high data fidelity with zero idle infrastructure costs..

Project Objectives

- **Implement Event-Driven Architecture:** Design a reactive system where data processing is triggered instantaneously by an `s3:ObjectCreated` event, eliminating the need for inefficient, time-based polling or cron jobs.

- **Automate Data Sanitization at the Edge:** Utilize Python and the `pandas` data manipulation library to programmatically execute critical cleaning transformations—specifically handling null values, eliminating row-level duplicates, and standardizing schema headers—without any manual human intervention.

- **Enforce Cloud Security and Governance:** Apply the principle of least privilege by configuring strict AWS Identity and Access Management (IAM) execution roles. Ensure that compute modules only have permission to read from the raw source and write to the clean destination, mitigating the blast radius of potential security breaches.

- **Optimize for Cost-Efficiency:** Engineer the entire architectural footprint to operate strictly within the AWS Free Tier. By utilizing AWS Lambda instead of permanent EC2 instances, the project will demonstrate a modern "scale-to-zero" cost model where billing is calculated per millisecond of actual execution time.

- **Establish Telemetry and Auditability:** Integrate Amazon CloudWatch to establish an automated logging framework, ensuring every data transformation is tracked, measured, and auditable for debugging purposes.

Proposed Methodology & Architecture

The proposed system follows a decoupled, stateless, and serverless workflow designed for maximum efficiency:

1. **Data Ingestion (The Trigger):** A raw, unformatted CSV payload is uploaded to a securely designated "Raw Data" Amazon S3 bucket.
2. **Event Notification:** The S3 bucket automatically detects the PUT request and broadcasts an event notification directly to the compute layer, passing the exact bucket name and object key.
3. **Serverless Computation:** An AWS Lambda function is provisioned on-demand. Equipped with an AWS-managed `pandas` layer, the function downloads the file into ephemeral storage and loads it into memory as a DataFrame.
4. **Transformation Logic:** The Python script executes a deterministic series of cleaning operations: it isolates and drops incomplete records (NaN/Nulls), scrubs duplicate entries, normalizes text casing, and formats column headers into a standardized schema.

5. **Data Export:** The sanitized DataFrame is converted back into a CSV format in-memory and dynamically written to a logically separated "Clean Data" Amazon S3 bucket, preventing cyclic trigger loops.
6. **Telemetry Logging:** Throughout the execution lifecycle, the Lambda function streams real-time metrics—including execution duration, pre- and post-cleaning row counts, and error stack traces—directly to Amazon CloudWatch log streams.

Technology Stack

- **Compute: AWS Lambda.** Chosen over Amazon EC2 to eliminate idle capacity costs and server patching. Lambda provides sub-second scaling and native integration with S3 events.

- **Storage: Amazon S3 (Simple Storage Service).** Chosen over block storage (EBS) due to its virtually infinite scalability, high durability (11 nines), and native event-notification capabilities.

- **Security: AWS IAM (Identity and Access Management).** Utilized to construct granular, role-based access control (RBAC) policies binding the compute layer to specific storage prefixes.

- **Monitoring: Amazon CloudWatch.** Provides centralized log aggregation and metric tracking without requiring third-party monitoring agents.

- **Language & Libraries:** Python 3.x, `pandas` (for high-performance data manipulation), and `boto3` (the AWS SDK for Python for programmatic service interaction).

Project Scope and Limitations

- **In Scope:** The automated ingestion, transformation, and storage of small-to-medium CSV payloads. The implementation of secure IAM policies, S3 event triggers, and foundational infrastructure telemetry.

- **Out of Scope (Limitations):** The processing of massive enterprise datasets (e.g., payloads exceeding 5 GB). Because the `pandas` library operates entirely in-memory, datasets exceeding AWS Lambda's strict 10 GB memory limit or 15-minute execution timeout will trigger an Out-Of-Memory (OOM) error. Advanced data imputation techniques (e.g., using machine learning to predict missing values) are also excluded, as the primary academic focus remains on foundational cloud infrastructure methodologies rather than advanced data science.